Foundations of Computing I
Summary

Simon Schuhmacher

UZH FS2026

Contents

1 Propositional Logic

Basic Logical Equivalences

Commutative Law

- $p \wedge q \equiv q \wedge p$
- $p \vee q \equiv q \vee p$

Associative Law

- $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
- $(p \vee q) \vee r \equiv p \vee (q \vee r)$

Distributive Law

- $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
- $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

Identity Law

- $p \wedge \mathbf{t} \equiv p$
- $p \vee \mathbf{c} \equiv p$

Negation Law

- $p \vee \sim p \equiv \mathbf{t}$
- $p \wedge \sim p \equiv \mathbf{c}$

Double Negation Law

- $\sim(\sim p) \equiv p$

Idempotent Law

- $p \wedge p \equiv p$
- $p \vee p \equiv p$

Universal Bound Law

- $p \vee \mathbf{t} \equiv \mathbf{t}$
- $p \wedge \mathbf{c} \equiv \mathbf{c}$

De Morgan's Law

- $\sim(p \wedge q) \equiv \sim p \vee \sim q$
- $\sim(p \vee q) \equiv \sim \wedge \sim q$

Absorption Law

- $p \vee (p \wedge q) \equiv p$
- $p \wedge (p \vee q) \equiv p$

Negations of t and c

- $\sim\mathbf{t} \equiv \mathbf{c}$
- $\sim\mathbf{c} \equiv \mathbf{t}$

XOR

- $p \oplus q \equiv (p \vee q) \wedge \sim(p \wedge q)$
- $p \oplus q \equiv (a \wedge \sim b) \vee (\sim a \wedge b)$

Implication (If-Then)

- $p \to q \equiv \sim p \vee q$

Equivalence (Biconditional If and Only if)

- $p \leftrightarrow q \equiv (p \to q) \wedge (q \to p)$

2 Digital Logic Circuits

Disjunctive Normal Form (DNF)

"OR of ANDs"

From truth table to DNF

1. Identify the rows whose output is 1
2. AND together all variables in the same row (negate the variable when 0)
3. OR together the 1-rows

Conjunctive Normal Form (CNF)

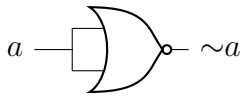
"AND of ORs"

From truth table to CNF

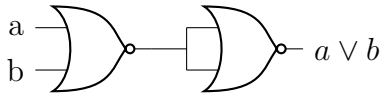
1. Identify the rows whose output is 0
2. OR together all variables in the same row (negate the variable when 1)
3. AND together the 0-rows

NOR Gate

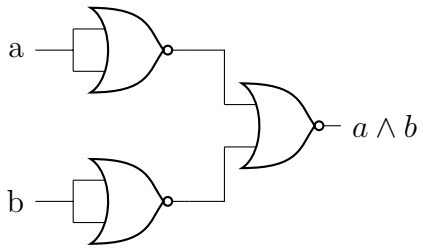
NOT Using NOR



OR Using NOR

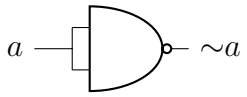


AND Using NOR

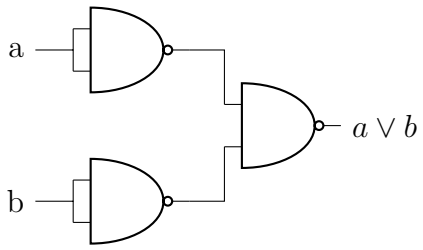


NAND Gate

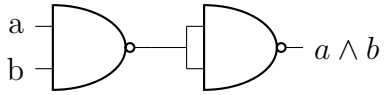
NOT Using NAND



OR Using NAND



AND Using NAND



3 Induction and Recursion

Proof by Mathematical Induction

To prove that a property $P(n)$ is **true** for every integer $n \geq b$:

1. **Base Case**: Show that $P(b)$ is true
2. **Inductive Hypothesis**: Assume $P(k)$ true for any $k \geq b$
3. **Inductive Step**: **Show** that $P(k) \to P(k+1)$ **is true**
4. **Conclusion**: If $P(b)$ is true and $P(k) \to P(k+1)$ is true, conclude that $P(n)$ is true for all $n \geq b$

Proof by Strong Mathematical Induction

- Prove recursive problems with several base cases (e.g., recurrence relations with order larger than 1)
- Makes a stronger hypothesis than the principle of Mathematical Induction:
  - The **base step** may contain proofs for **several initial values**, rather than just the base case
  - In the inductive step, the trust of $P(k)$ is assumed not just for an arbitrary $k \geq b$ but for **all values through** k

Let b, b be two integers with $a \leq b$. To prove that a property $P(n)$ is **true** for every integer $n \geq a$:

1. **Base Case**: Show that $P(n)$ is true for all base cases from a to b, that is: $P(a) \wedge P(a+1) \wedge \cdots \wedge P(b)$
2. **Inductive Hypothesis**: For any $kgeqb$, assume $P(i)$ **is true for** $i = a, \ldots, b, \ldots, k$
3. **Show** that $P(k+1)$ **is true**
4. **Conclusion**: If 1 and 2 are verified, conclude that $P(n)$ is true for all $n \geq a$

Method of the Loop Invariant to Prove Algorithm Correctness

For every $n \geq 0$, let $P(n)$ be the loop invariant for a loop. **If the following four properties are true, then the loop is correct** with respect to its pre- and post-conditions:

1. **Basic property**: the loop invariant is true before the loop starts, i.e., $P(0)$ is true before the loop starts. ($P(0)$ is $P(n)$ when $n = 0$). → "The loop invariant is true before the loop starts (i.e., base case is true)"
2. **Inductive property**: for any integer $k \geq 0$, if **both the guard G and the loop invariant** $P(k)$ are true before an iteration of the loop, then $P(k+1)$ is true after iteration. → "It remains true after every iteration (i.e., $P(k) \rightarrow P(k+1)$)"
3. **Eventual Falsity of the Guard**: after a finite number of iterations of the loop, the guard G becomes false. → "The loop eventually stops (i.e., that the loop condition eventually turns false)"
4. **Correctness of the post-condition**: if N is the number of iterations after which G is false and $P(n)$ is true, then the post-condition is satisfied. → "At the point when the loop ends, you need to verify that the **loop invariant** is true"

4 Functions and Convolution

Laws of Exponents

If b and c are any positive real numbers and u and v are any real numbers, the following laws of exponents hold true:

- $b^u b^v = b^{u+v}$
- $(b^u)^v = b^{uv}$
- $\frac{b^u}{b^v} = b^{u-v}$
- $(bc)^u = b^u c^u$

Laws of Logarithms

For any positive real numbers b, c and x with $b \neq 1$ and $c \neq 1$:

- $\log_b(xy) = \log_b x + \log_b y$
- $\log_b(\frac{x}{y}) = \log_b x - \log_b y$
- $\log_b(x^a) = a \log_b x$
- $\log_c x = \frac{\log_b x}{\log_b c}$

1D Convolution

$f(n)$, $g(n)$, $y(n)$: $\mathbb{Z} \rightarrow \mathbb{R} \quad \forall n \in \mathbb{Z}$

1D convolution between f and g, denoted with $y(n) = (f * g)(n) = \sum_{k=-\infty}^{+\infty} f(k)g(n-k)$

- f: input function
- g: kernel or filter

To convolve two functions f and g: $f * g$

1. Flip g horizontally ($-k$)
2. Slide it over f by adjusting n and compute the sum of the products, starting at the first overlap
3. Continue until the last overlap
4. Calculate $y(n)$ for using all n-values used to slide g over f

Note: Since the operation is commutative, sometimes it might be more convenient to compute $g * f$

Convolution Table Approach

1. Create vector representations of f and g
2. Identify *min index* and *max index* → min and max non-zero values on the x-axis for f and g
3. Create a table with f as columns from *min index* to *max index* and g as rows from *min index* to *max index*
4. Multiply the values
5. Create $y(n)$ by summing the diagonals
6. $n = 0$ at the diagonal where *row index* + *column index* $= 0$ (and equivalent for other n-values)

Properties of Convolution

- **Commutative Law**: $f * g = g * f$
- **Associative Law**: $(f * g) * h = f * (g * h)$

- **Associative Law with Scalar Multiplication**:
  $(\lambda f) * g = f * (\lambda g) = \lambda(f * g)$
- **Distributive Law**:
  $f * (g + h) = (f * g) + (f * h)$
- **Convolution with a Dirac**: $f * \delta = f$

Why We Flip the Function in the Definition of Convolution

$(f * g)(n) = (g * f)(n) \Rightarrow$
$\sum_{k=-\infty}^{+\infty} f(k)g(n-k) = \sum_{k=-\infty}^{+\infty} g(k)f(n-k)$

- Without flipping, convolution would not be commutative

Dirac Function

Also called *impulse function*. The result of the convolution of a function $f(n)$ with a Dirac is the function itself: $(f * \delta)(n) = f(n)$

$$\delta(n) = \begin{cases} 1 & n = 0 \\ 0 & \text{elsewhere} \end{cases}$$

or

$$\delta(n-i) = \begin{cases} 1 & n = i \\ 0 & \text{elsewhere} \end{cases}$$

Note that $f(n) * \delta(n-i)$ shifts $f(n)$ by i to the right.

2D Convolution

Let F, G, Y be **functions** defined on the infinite set $\mathbb{Z}^2$: $F(i,j)$, $G(i,j)$, $Y(i,j)$:
$\mathbb{Z}^2 \to \mathbb{R} \quad \forall (i,j) \in \mathbb{Z}^2$

We define the 2D convolution between f and g: $Y(i,j) = (F * G)(i,j) =$
$\sum_{u=-\infty}^{+\infty} \sum_{v=-\infty}^{+\infty} F(u,v)G(i-u, j-v)$

- f: input image
- g: kernel or filter

Box Filters

1. Flip the filter in both dimensions (= 180 deg turn)
2. Slide the filter over the image, starting in the top right
3. Compute the sum of products (replace each pixel with a weighted sum of its neighbors)
4. Continue towards the right until the end of the row
5. Move to the next row, starting from the left again

Note: If we don't want to change the intensities, the sum of all elements in the filter should be 1.

Example Low-Pass Filter

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

Example High-Pass Filter

0	1	0
1	−4	1
0	1	0

Example Horizontal (x) Axis Filter

−1	0	1
−1	0	1
−1	0	1

Example Vertical (y) Axis Filter

−1	−1	−1
0	0	0
1	1	1

Boundary Issues

- **Zero-padding** of the input image boundaries to achieve the **same size** of the input and output image.

- **No zero-padding** of the input image results in a **smaller size** of the output image.

Filters of Even Side Length

- If our filter has even side length, we need to determine the center out of 4 options. By convention, we chose the point at the bottom right (option 4)
- If we add zero-padding, this needs to be adapted according to where the center is chosen

	1	2	
	3	**4**	

5 Relations and Cryptography

Reflexivity, Symmetry, and Transitivity

Let R be a relation on a set A

1. R is **reflexive** if, and only if, for all $x \in A, x \; R \; x$
2. R is **symmetric** if, and only if, for all $x, y \in A$, ***if*** $x \; R \; y$ then $y \; R \; x$
3. R is **transitive** if, and only if, for all $x, y, z \in A$, ***if*** $x \; R \; y$ and $y \; R \; z$ then $x \; R \; z$

A relation that satisfies all these three properties is called an **Equivalence Relation**.

Transitive Closure

To make a relation transitive that does not contain certain **ordered pairs**, we need to add some ordered pairs. The relation R^t obtained by adding the **minimum number of ordered pairs** to ensure transitivity is called the **transitive closure** of the relation.

The Relation Induced by a Partition

- A **partition** of a set A is a collection of mutually disjoint subsets A_i whose union is A
- Given a partition, the relation **induced** by the partition of A is a relation where all the elements within each subset A_i of the partition are related to one another and themselves
- A relation induced by a partition is always **reflexive**, **symmetric**, and **transitive**, therefore it is always an **equivalence relation**

Equivalence Classes of an Equivalence Relation

- Given an equivalence relation on a certain set A and any particular element a in A, the subset $[a]$ of all elements related to a under R is called the **equivalence class** of a: $[a] = \{x \in A \mid x \; R \; a\}$
- **Distinct equivalence classes** of an *equivalence relation* form a **partition** of A
- Any element of an equivalent class is called **Class Representative**
- E.g., for $\{0, 3, 4\}, \{1\}, \{2\}$, the **distinct equivalence classes** of R are: $[0], [1], [2]$ (or alternatively $[3], [1], [2]$ or $[4], [1], [2]$)
  - $[0] = [3] = [4]$ because they represent the same equivalence class

Congruence Modulo n

Let a, b, and n be any integers and suppose $n > 1$. The following statements are all equivalent

1. $n \mid (a - b)$
2. $a \equiv b \bmod n$
3. $a = b + kn$ for some integer k

4. a and b have the same (nonnegative) remainder when divided by n
5. $a \bmod n = b \bmod n$

Properties of Congruence Modulo n

1. $(a + b) \bmod n = [(a \bmod n) + (b \bmod n)] \bmod n$
2. $(a - b) \bmod n = [(a \bmod n) - (b \bmod n)] \bmod n$
3. $(a \cdot b) \bmod n = [(a \bmod n) \cdot (b \bmod n)] \bmod n$
4. $(a^b) \bmod n = (a \bmod n)^b \bmod n$
5. $-a \bmod b = (b - (a \bmod b)) \bmod b$

If you first perform addition, subtraction, multiplication of integers and then **reduce the result modulo** n, you obtain the same result as when the operation is performed on the reduced version of the integers.

Finding an Inverse Modulo n

The modular inverse of an integer a is an integer x such that:

- $a \cdot x \equiv 1 \pmod{n}$
- That means: $(a \cdot x) \bmod n = 1$
- *Example:* Compute the inverse of 3 modulo 7: We want to determine x such that $3x \equiv 1 \pmod{7}$. An inverse is $5 \equiv 1 \pmod{7}$. Other possibilities are e.g., 12, -2.
- The **inverse** of an integer a modulo n exists if and only if a and n are **relatively prime**.
- Two integers a and b are **relatively prime** if and only if $\gcd(a, b) = 1$

Bézout's Theorem

If a and b are positive integers, then there exist integers s and t such that $\gcd(a, b) = \boldsymbol{s}a + \boldsymbol{t}b$

Euclidean Algorithm

Let A and B be two integers with $A > B \geq 0$, Euclid's algorithm is defined by the following recursive function:

$$\gcd(A, B) = \begin{cases} A & \text{if } B = 0 \\ \gcd(B,\ A \bmod B) & \text{if } B > 0 \end{cases}$$

$A \bmod B$ denotes the **remainder** of the division of A by B.

RSA Cryptography

- The plaintext M is converted into a ciphertext C: $C = M^e \bmod S$
- S and e are the **public keys**, i.e., anyone can use them to encrypt their messages
- The plaintext M for a ciphertext C is then recovered: $M = C^d \bmod S$
- Where d is the **private key**, i.e., it is secret, and only the recipient knows it
- For RSA to work $M = (M^e)^d \bmod S$ must hold for all positive integers $M < S$
- This holds if:
  - $S = p \cdot q$ where p and q are **prime numbers**
  - e and the product $(p - 1)(q - 1)$ are **relatively prime**, i.e., $\gcd(e, (p - 1)(q - 1)) = 1$
  - d is a **positive inverse** of $e \bmod (p - 1)(q - 1)$, i.e., $ed \equiv 1 \pmod{(p - 1)(q - 1)}$
- RSA works because it is easy to generate two large prime numbers p and q, but it is hard to factor their product S into pq

Bézout's Identity

For any non-zero integers a and b with $\gcd(a, b) = d \quad \exists x, y$ (Bézout coefficients) such that $ax + by = d$

6 Graphs and Trees

Graphs: Terminology

- **Nodes** or **vertices**: Dots in the graph (can be isolated)
- **Edges**: Connect vertices (edges must always be connected to vertices)
- **Directed graphs**: Graphs where edges have a direction
- **Complete graphs**: Graphs where all pair combinations of vertices are connected by edges, has $\frac{n(n-1)}{2}$ edges, where n is the number of vertices
- **Degree of a vertex** v: The number of segments that start or end at v. The degree of a **loop** is 2.
- **Total degree** of a graph: The sum of the degrees of all its vertices, the **total degree of a graph is always twice the number of edges**.
- A graph is **connected** if it is possible to find a route from any vertex to any other vertex.
- A graph is **not connected** if, and only if, there are two vertices that are not connected by any route.
- **Connected components**: The sub-graphs of a graph which are connected.

Routes: Definitions

- Route (also called *walk* or *travel*) in a graph is accomplished by moving from one vertex to another along a sequence of adjacent edges.
- **Trails**: routes that do not have a repeated edge.
- **Paths**: *trails* that do not have repeated vertices.
- **Circuits**: *trails* that start and end at the same vertex.
- **Simple circuits**: *circuits* that only pass through each vertex of the circuit once, except for the start and end vertices.

Euler Circuit

- *Circuit* that passed through all the **vertices at least once** and passes through all the **edges only once**
- A graph has an Euler Circuit **if, and only if** the graph is **connected** and **every vertex** of the graph has **even degree** (because the total number of arrivals and departures from each vertex must be a multiple of 2).

Hamiltonian Circuits

- *Circuit* that passes through all the **vertices only once** (except for the start and end vertices). Since it is a circuit, it has no repeated edges.
- Does not need to include every edge.
- A Hamiltonian Circuit is a *simple circuit* that **passes through all vertices of the graph** (unlike in the definition of a "normal" simple circuit, where it is not required to pass through all vertices).
- There is no simple criterion for determining whether a graph has a Hamiltonian circuit, nor is there an efficient algorithm for finding such a circuit

If a graph G **contains a Hamiltonian Circuit** H, then:

1. G must be connected
2. H must contain all vertices
3. The number of edges of H must equal the number of its vertices
4. **Every vertex of** H **must have exactly degree** 2

If a graph G does not have a sub-graph H with these properties, **then** G **does not have a Hamiltonian circuit**.

- To check whether a Hamiltonian Circuit exist, check for all vertices that have exactly two edges and mark them
- Check for edges which cannot be taken (since already two edges are marked for the vertex)
- Check for vertices which cannot be visited because of this

Traveling Salesman Problem

- For a **complete graph** with n cities, there are $\frac{(n-1)!}{2}$ unique *Hamiltonian Circuits*.

Matrices and Directed Graphs

- **Definition**: Let G be a directed graph with ordered vertices $v_1, v_2, \ldots, v_n$. The **adjacency matrix of** G is the $n \times n$ matrix $A = a_{ij}$ over the set of nonnegative integers such that a_{ij} = the number of arrows from v_i to v_j for all $i, j = 1, 2, \ldots, n$.
- i is the **index of the row** and marks the **starting point**
- j is the **index of the column** and marks the **destination**

Matrices and Undirected Graphs

- The adjacency matrix of an undirected graph is symmetric, i.e., $a_{ij} = a_{ji}$
- *Note:* A directed graph might also have a symmetric adjacency matrix (if for every arrow between two distinct vertices there is an arrow back).

Computing the Number of Distinct Routes of Length n

- The number of distinct routes of length n in an adjacency matrix A is A^n
- **Matrix multiplication**:
  - $a_{1,1} = a_{1,1} \cdot a_{1,1} + a_{1,2} \cdot a_{2,1} + a_{1,3} \cdot a_{3,1}$
  - $a_{1,2} = a_{1,1} \cdot a_{2,1} + a_{1,2} \cdot a_{2,2} + a_{1,3} \cdot a_{2,3}$
  - $\ldots$
  - $a_{3,3} = a_{3,1} \cdot a_{1,3} + a_{3,2} \cdot a_{2,3} + a_{3,3} \cdot a_{3,3}$

Isomorphism of Graphs

- A graph G with a vertex set $V(G)$ and an edge set $E(G)$ is **isomorphic** to a graph G' with a vertex set $V(G')$ and an edge $E(G')$ if, and only if, there exist one-to-one correspondences $g : V(G) \to V(G')$ and $h : E(G) \to E(G')$ that preserve the edge-endpoint functions of G and G' in the sense that for all $v \in V(G)$ and $e \in E(G)$, v is an endpoint of $e \Leftrightarrow g(v)$ is an endpoint of $h(e)$
- I.e., G' has other names for the same vertices and edges than G (there exists a mapping from vertices in G to vertices in G')

Trees

- A *connected graph* that does **not contain any *circuits***
- A tee with n vertices has $n - 1$ edges

Rooted Trees

- A tree in which **one vertex** is designated (i.e., chosen) as the **root**
- There is a **unique path from the root** to **any other vertex** v
- **Level of a vertex**: Number of edges along the unique path between it and the root
- **Height of the tree**: Maximum level of the tree
- **Leaves**: Vertices without children
- **Internal vertices**: All vertices which are not *leaves*

Binary Tree

- A *rooted tree* where **each vertex has at most two children**

- **Perfect binary tree**: Each *internal vertex* has **exactly two children**, and all **leaves** are at the **same level**
- Let h be the **height** of the tree and t the number of its **terminal vertices**
  - For a **generic binary tree**: $t \leq 2^h$
  - For a **perfect binary tree**: $t = 2^h$

Spanning Trees

- A *spanning tree* for a graph is a **tree that contains every vertex of the graph**
- **Every connected graph** has a spanning tree
- All spanning trees for a graph have the **same number of edges**
- To get the spanning trees of a graph that contains a *circuit*, remove any of the edges of the circuit to get a tree. There might be multiple possibilities.

Minimum Spanning Tree

- **Weighted graph**: A graph whose edges are labeled with numbers (weights)
- **Minimum spanning tree**: A spanning tree for which the **sum of the weights of all the edges** is **as small as possible**
- A *complete graph* with n vertices has n^{n-2} spanning trees

Kruskal's Algorithm

1. Mark the **edge with the minimum weight**
2. Add the next unmarked edge with minimum weight **only if its addition does not generate a circuit**
3. **Repeat from 2** until there are no more edges to add

Prim's Algorithm

1. Pick an **arbitrary** vertex
2. Mark the edge connected to it that has the **minimum weight** and mark its connected vertex as visited
3. Check all vertices visited so far **which still "see" unvisited vertices** and mark the one adjacent edge with the **minimum weight** and mark its connected vertex as visited
4. **Repeat from 3** until all vertices have been visited

Kruskal's vs. Prim's

- Both algorithms return a **minimum spanning tree**, thus both are **optimal**
- **Kruskal's** runs **faster for sparse graphs**
- **Prim's** runs faster for **dense graphs** (given two graphs with the same number of vertices, one graph is said to be **denser** than the other if it has **more edges**)

Dijkstra's Shortest Path Algorithm

- Find the shortest path from a **start vertex** to **any other vertex** in a weighted graph with positive weights
- *Note:* Dijkstra's finds a spanning tree but this thee **may not be a minimum spanning tree**

1. Define the start vertex S (this is usually given)
2. Assign to all its **adjacent** vertices the cost to reach them from S and to all non-adjacent vertices an **infinite cost**
3. Check all vertices visited so far that still "see" unvisited vertices, then visit the vertex with the minimum cost and update the cost of all its adjacent unvisited vertices with the total cost to reach them from S passing through the current vertex only if the cost is less than the previous cost

4. **Repeat from 3** until all vertices have been visited